

IV. Ko'p Oqimli (Multi-threading) va Asinxron Tizimlar

Real vaqtdagi chat dasturlari uchun server bir vaqtning o'zida yuzlab foydalanuvchilarga xizmat ko'rsatishi shart. Ushbu bo'limda parallelizm asoslarini o'rganamiz.

4.1. Threading moduli: Bir vaqtning o'zida bir nechta mijozni qabul qilish

Oldingi darslardagi serverlarimiz "Iterative" (ketma-ket) edi: bitta mijoz ulanib, ishini tugatmaguncha keyingisi kutib turardi. **Threading** (oqimlar) yordamida har bir yangi ulanish uchun alohida "ishchi" (thread) ajratamiz.

- **Asosiy oqim (Main Thread):** Yangi mijozlarni kutadi (`accept()`).
- **Ishchi oqim (Worker Thread):** Ulanish sodir bo'lgach, aynan shu mijoz bilan muloqot qiladi.

Amaliy namuna:

```
import socket
import threading

def handle_client(conn, addr):
    print(f"[NEW CONNECTION] {addr} ulandi.")
    connected = True
    while connected:
        data = conn.recv(1024)
        if not data: break
        print(f"[{addr}] {data.decode('utf-8')}")
        conn.sendall("Xabar yetib bordi".encode('utf-8'))
    conn.close()

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('127.0.0.1', 5050))
server.listen()

print("[STARTING] Server ishga tushdi...")
while True:
    conn, addr = server.accept()
    # Har bir mijoz uchun alohida oqim yaratish
    thread = threading.Thread(target=handle_client, args=(conn, addr))
    thread.start()
    print(f"[ACTIVE CONNECTIONS] {threading.active_count() - 1}")
```

4.2. Race Conditions va Lock mexanizmi

Ko'p oqimli tizimlarda bitta muammo bor: agar ikkita oqim bir vaqtning o'zida bitta umumiy o'zgaruvchini (masalan, umumiy chat tarixi yoki foydalanuvchilar ro'yxati) o'zgartirmoqchi bo'lsa, ma'lumotlar buziladi. Bu **Race Condition** (poyga holati) deyiladi.

- **Lock (Qulflab):** Bir vaqtning o'zida faqat bitta oqim ma'lumotni o'zgartirishi uchun uni "qulflab" qo'yish.

```
import threading

lock = threading.Lock()
chat_history = []

def update_history(message):
    with lock: # Faqat bitta thread kiradi
        chat_history.append(message)
```

4.3. Asyncio kutubxonasi: High-load tizimlar

Threading resurs talab qiladi (har bir oqim RAM egallaydi). Minglab ulanishlar uchun **Asyncio** (Asinxron kirish-chiqish) ko'proq mos keladi.

- **Asinxronlik:** Dastur ma'lumot kelishini kutib o'tirmaydi, boshqa ishlarni bajarib turadi. Ma'lumot tayyor bo'lganda unga qaytadi.
 - **Event Loop:** Barcha hodisalarni (events) bitta oqimda boshqaruvchi markaz.
-

4.4. Kiberxavfsizlik: Thread Exhaustion Attack

Agar serveringizda ulanishlar soni cheklanmagan bo'lsa, buzg'unchi minglab ulanishlarni ochib, server xotirasini (RAM) to'ldirib yuborishi mumkin.

Himoya choralari:

1. **Thread Pool:** Bir vaqtning o'zida ochiladigan oqimlar sonini (masalan, maksimal 100 ta) cheklash.
2. **Keep-alive Timeout:** Ma'lumot yubormayotgan oqimlarni majburiy yopish.

In []: